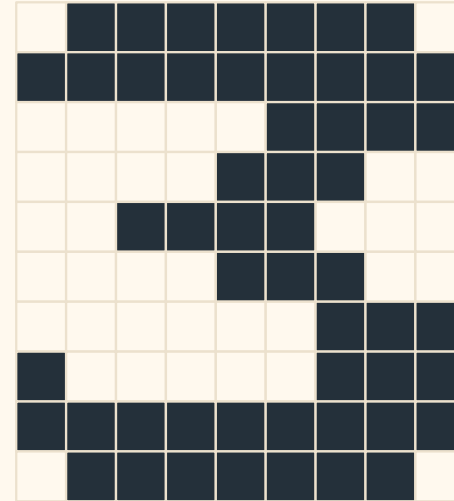


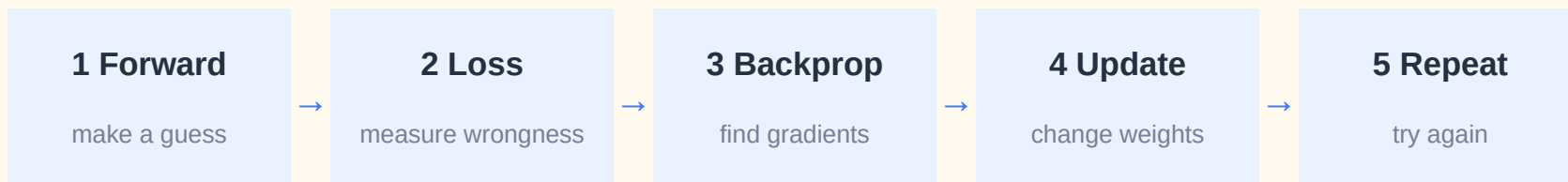
What number is this?

Your phone reads it instantly.
In one hour you build the thing
that can - one Python file, from scratch.



Guess. Measure. Change. Repeat.

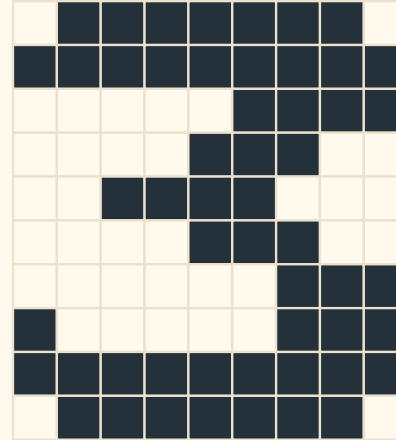
- A network starts with random weights, so its first guesses are bad - that is where learning begins.
- Each round measures how wrong the guess was, finds which weights caused the error, and nudges every weight a little.



Every line of code today belongs to one of these five moves.

A picture is 784 numbers

- A 28 x 28 grayscale image: one brightness value per pixel, 0 to 255.
- Flatten the square into one row of 784 numbers - no pixel is lost.
- Divide by 255 so every value lands between 0 and 1.



```
x = image.view(-1, 784) / 255.0
```

Weighted sums, plus a bias

- A layer multiplies each input by an adjustable weight, adds the products, then adds a bias - producing a new set of numbers.
- The key word is "adjustable": learning is nothing more than changing these weights.

$(\text{images}, 784) @ (784, 128) + (128) \rightarrow (\text{images}, 128)$

@ does every multiply-and-add at once. Inside dims match; outside dims survive.

Build one learned layer

Random weights, zero biases.
Ask PyTorch to track them.

forward() makes the guess.
parameters() hands the loop
every value it may change.

```
class Linear:
    def __init__(self, in_features, out_features):
        std = (2 / in_features) ** 0.5
        self.weights = torch.randn(
            in_features, out_features) * std
        self.bias = torch.zeros(out_features)

        self.weights.requires_grad_()
        self.bias.requires_grad_()

    def forward(self, x):
        return x @ self.weights + self.bias

    def parameters(self):
        return [self.weights, self.bias]
```

Checkpoint 1 - save the file, run the cell. Expect (3, 784) -> (3, 128).

Add a bend

- Stack linear layers with nothing between them and the whole stack is still just one linear layer.
- Digits are messy; the model needs bends to draw richer boundaries.

```
class ReLU:  
    def forward(self, x):  
        return x.clamp(min=0)
```

Read it as

Below zero becomes zero.

Everything else passes through.

[-2, 0, 3] -> [0, 0, 3]

784 -> 128 -> 128 -> 10

Linear, ReLU, Linear, ReLU, Linear. The ReLUs change values, not dimensions.

parameters() adds up
to six tensors:
three weights,
three biases.

```
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        self.layer1 = Linear(input_size, hidden_size)
        self.layer2 = Linear(hidden_size, hidden_size)
        self.layer3 = Linear(hidden_size, output_size)
        self.relu1 = ReLU()
        self.relu2 = ReLU()

    def forward(self, x):
        x = self.layer1.forward(x)
        x = self.relu1.forward(x)
        x = self.layer2.forward(x)
        x = self.relu2.forward(x)
        x = self.layer3.forward(x)
        return x
```

Checkpoint 2 - four images become ten scores each. Expect (4, 784) -> (4, 10).

Ten scores -> one guess

- Ten numbers out, one per digit. The largest is the model's guess (argmax).
- A raw score is called a logit.
- `prediction.softmax(dim=1)` turns the ten into probabilities that add to 1 - same guess, readable confidence.

Cross-entropy will compare all ten scores with the correct digit.

Load MNIST

Examples for learning,
separate examples for testing.

Flatten to 784.
Scale 0-255 down to 0-1.
Keep each digit as the target.

```
train = torchvision.datasets.MNIST(  
    root="./data", train=True, download=True)  
test = torchvision.datasets.MNIST(  
    root="./data", train=False, download=True)  
  
x = train.data.float().view(-1, 784) / 255.0  
target = train.targets  
x_test = test.data.float().view(-1, 784) / 255.0  
target_test = test.targets
```

One number, then every gradient

- `loss = cross_entropy(prediction, target)` is high when the right digit scores poorly, low when it wins - the single signal learning reduces.
- `loss.backward()` walks the remembered operations in reverse and reports how each weight contributed to the error.

```
loss <- scores <- layers <- every weight and bias
```

A gradient is a direction: changing this value that way would raise the loss.

One small step, then clear

Each gradient points
toward more error,
so subtract a little
of it from each weight.

```
loss.backward()

with torch.no_grad():
    for parameter in model.parameters():
        parameter -= learning_rate * parameter.grad

for parameter in model.parameters():
    parameter.grad.zero_()
```

Read it as

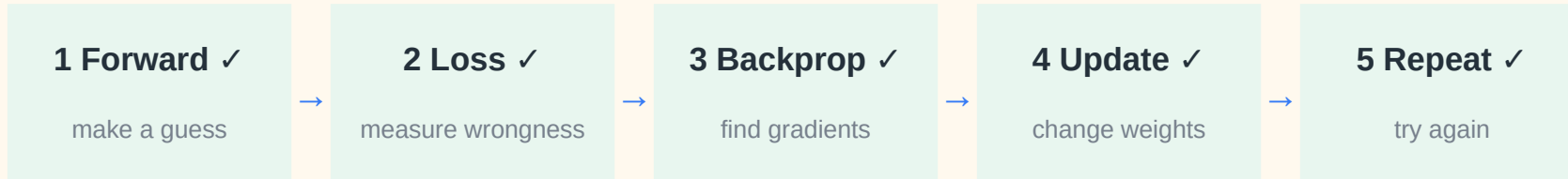
Step opposite the gradient, a little.

no_grad(): this is the change itself.

Clear the grads, or the next round mixes old with new.

Checkpoint 3 - one manual step makes a fixed problem's loss smaller.

Nothing is hidden



```
1 prediction = model.forward(x)
2 loss = cross_entropy(prediction, target)
3 loss.backward()
4 parameter -= learning_rate * parameter.grad
5 parameter.grad.zero_() -> next epoch
```

Ten rounds, then sixty

~2.48

starting loss

~1.69

loss after 10 epochs

~65%

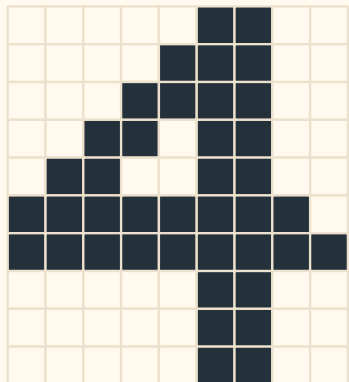
test accuracy, 10 epochs

Change epochs 10 -> 60: loss ~0.48, test accuracy ~88%. Same architecture, more rounds.

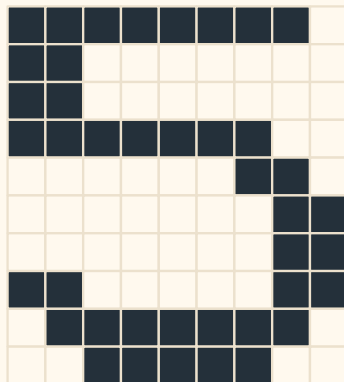
Blind guessing is 10%. Read the trend: loss down, accuracy up.

Illustrative seeded output; facilitator, use the numbers you measured on the event machine.

It matches pixels - it does not know digits



guess 9 · truth 4



guess 3 · truth 5

It learned pixel patterns that usually work. When a 4 is drawn like a 9, it has nothing else to fall back on.

Keep the file. Change one thing.

- 1 Download mnist_network.py
- 2 Pick one experiment
- 3 Predict what will happen
- 4 Change one value
- 5 Run and compare

Try one:
epochs 10 -> 60
learning rate 0.1 -> 0.01
hidden size 128 -> 32
break the init (std = 1.0)
layer1_weight_grid(model)

You built the layers, the gradients, and the update. The high-level PyTorch tools package the same machinery.